

Session 1.3: Tabular Data Wrangling Part 1: NumPy Arrays, Pandas DataFrames, and CSV Ingestion

1:2 DELIVERY RHYTHM

Applied Learning

60 minutes of focused instruction followed by 120 minutes of guided coding and debugging.

SESSION OUTCOME

Operational Competency

Students apply the session concept to a small, verifiable business dataset.

Concepts for This Session

CONCEPT 1

Vectorized Computation

Executing mathematical operations across entire numeric arrays or data columns simultaneously in compiled C memory, replacing slow Python row-by-row loops.

CONCEPT 2

Pandas Series

A one-dimensional labeled array in Pandas capable of holding any data type, representing a single column in an enterprise tabular dataset.

CONCEPT 3

Pandas DataFrame

A two-dimensional, size-mutable tabular data structure with labeled axes (rows and columns), serving as the primary analytical workhorse for commercial data.

CONCEPT 4

Tabular CSV Ingestion

Reading structured comma-separated corporate ledgers directly into memory using `pd.read_csv()`, parsing data types and column headers automatically.

How the Concept Works in Practice

FOUNDATION 1

Vectorized Array Math vs. Row-by-Row Iteration

While Python for loops are intuitive, calculating metrics across 500,000 retail records using a loop is computationally slow. NumPy and Pandas use vectorized execution, operating on entire memory blocks in parallel, reducing execution time from seconds to milliseconds.

FOUNDATION 2

The Two-Dimensional DataFrame Structure

Bridging spreadsheet grids into structured programmatic objects. A Pandas DataFrame consists of an Index (row identifiers), Columns (attribute headers), and Values (underlying two-dimensional NumPy array).

FOUNDATION 3

Explicit Data Types and Memory Management

Unlike spreadsheets where a cell can silently mix text and numbers, Pandas enforces explicit column data types (float64, int64, object, datetime64). This prevents silent financial corruption during aggregation.

FOUNDATION 4

Declarative Data Inspection

Mastering instantaneous programmatic diagnostics using `.info()` (data types and null counts), `.describe()` (descriptive summary statistics), and `.shape` (row and column dimensions).

Build, Test, and Diagnose in the IDE

[1] FILE EXPLORER

Organize Assets

Use the session dataset and keep scripts in a clear workspace.

[2] CODE EDITOR

Write Small Segments

Run each completed segment before expanding the program.

[3] AI AGENT CHAT

Diagnose Errors

Capture the traceback, request an explanation, and review the fix.

[4] INTEGRATED TERMINAL

Verify Execution

Confirm the final program completes with return code 0.

Script Demonstration 1: Tabular Data Wrangling Part 1: NumPy Arrays, Pandas DataFrames, and CSV Ingestion

```
import pandas as pd

df = pd.read_csv("data/session_01_03_dataset.csv")
print("Dataset Dimensions (Rows, Columns):", df.shape)
print("Column Data Types:\\n", df.dtypes)
print(df.head(3))
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 2: Tabular Data Wrangling Part 1: NumPy Arrays, Pandas DataFrames, and CSV Ingestion

```
df["Operating_Profit_USD"] = df["Monthly_Sales_USD"] -  
df["Operating_Cost_USD"]  
    df["Operating_Margin_Pct"] = (df["Operating_Profit_USD"] /  
df["Monthly_Sales_USD"]) * 100.0  
    print(df[["Store_ID", "Monthly_Sales_USD",  
"Operating_Margin_Pct"]].head(3))
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 3: Tabular Data Wrangling Part 1: NumPy Arrays, Pandas DataFrames, and CSV Ingestion

```
# Select high-revenue stores in the West region
west_high_sales = df.loc[(df["Region_Name"] == "West") &
(df["Monthly_Sales_USD"] >= 150000.0)]

# Impute missing satisfaction scores with median value
median_score = df["Customer_Satisfaction_Score"].median()
df["Customer_Satisfaction_Score"] =
df["Customer_Satisfaction_Score"].fillna(median_score)
print("Median Satisfaction Score Imputed:", median_score)
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Exercise 1 and Exercise 2

EXERCISE 1: GUIDED

Foundational Practice

Follow the lab protocol, run the script, and verify its output with the session dataset.

EXERCISE 2: DEBUGGING

Applied Challenge

Extend the program, diagnose an intentional error, apply the verified correction, and confirm return code 0.